

Sistema Interpretador de Instruções com Sensor de Distância e Atuadores

Placa: Arduino Mega 2560

Integrantes

Victor Alvarenga Hwang - 202208766005 - TA

Arthur Calebe Carvalho Da Silva - 202508449013 - TA

Matheus Cocenzo e Silva - 202107291052 - TA

1. Descrição Geral do Projeto

Este projeto implementa um **Sistema Interpretador de Instruções** baseado no conceito de **programa armazenado**, utilizando o Arduino Mega 2560 como plataforma física.

O sistema permite:

- Inserção de instruções via teclado matricial (modo LOAD)
- Armazenamento dessas instruções em memória de programa
- Execução controlada passo a passo (modo RUN)
- Processamento de dados provenientes de sensor ultrassônico
- Controle de dispositivos físicos (LEDs e buzzer)
- Simulação de uma arquitetura de computador com registradores e memória

O projeto aplica diretamente os conceitos de:

- Programa armazenado
- Ciclo de instrução
- ISA (Instruction Set Architecture)
- Unidade de Controle (UC)
- Unidade Lógica e Aritmética (ULA)
- Memória
- Registradores (PC, IR, ACC)
- Entrada e Saída

2. Elementos Arquiteturais Implementados

Elemento de Arquitetura	Implementação no Sketch
Memória de Dados	<code>int MEM[16];</code>
Program Counter (PC)	<code>int PC;</code>
Instruction Register (IR)	<code>byte IR;</code>
Acumulador (ACC)	<code>int ACC;</code>
Flag de Zero	<code>bool FLAG_Z;</code>
Controle de execução	<code>bool EXECUTANDO;</code>
Memória de Programa	<code>String PROGRAMA[16];</code>
Unidade de Controle	<code>executarProximaInstrucao(</code>
Unidade Lógica e Aritmética	Casos ADDK, SUBK, CMPK
Entrada	Teclado 4x4
Saída	LEDs, Buzzer, Serial Monitor
Sensor	HC-SR04

3.1 Modo LOAD

Ativado ao pressionar #.

Ações realizadas:

- Limpeza da memória de programa
- $PC = 0$
- Inicialização do ponteiro de carga
- Armazenamento das instruções digitadas

Cada instrução:

1. É digitada em formato decimal
2. Confirmada com tecla A
3. Armazenada no vetor PROGRAMA
4. Ponteiro é incrementado

As instruções **não são executadas** neste modo.

3.2 Modo RUN

Ativado ao pressionar B.

Comportamento:

- $PC = 0$
- Sistema entra em modo de execução
- A execução ocorre somente ao pressionar *
- Cada * executa exatamente **uma instrução**

4. Ciclo de Instrução

A função `executarProximaInstrucao()` implementa o ciclo:

1. Busca

```
instrucao = PROGRAMA[PC]
```

2. Decodificação

```
decodificarInstrucao()
```

3. Carga em IR

```
IR = opcode
```

4. Execução

```
executarInstrucao()
```

5. Atualização do PC

```
PC = PC + 1
```

6. Encerramento

Se `opcode = HALT` → `EXECUTANDO = false`

5. ISA Implementada

Decimal	Binário	Mnemônico	Descrição
0	0000	NOP	Não realiza operação
1	0001	READ	Lê sensor e armazena em ACC
2	0010	LOADK	Carrega constante em ACC
3	0011	ADDK	Soma constante ao ACC
4	0100	SUBK	Subtrai constante do ACC
5	0101	CMPK	Compara ACC com constante
6	0110	LEDON	Liga LED
7	0111	LEDOFF	Desliga LED
8	1000	BUZON	Liga buzzer
9	1001	BUZOFF	Desliga buzzer
10	1010	DISP	Exibe ACC
11	1011	ALERT	Resposta automática por distância
12	1100	BINC	Mostra opcode binário
13	1101	STORE	Armazena ACC em MEM[X]
14	1110	LOADM	Carrega MEM[X] em ACC
15	1111	HALT	Encerra execução

6. Funcionalidades (F01 a F11)

F01 — Entrada de instruções

Teclado 4x4 monta a string da instrução. Confirmação via tecla A.

F02 — Codificação

Função `decodificarInstrucao()` converte entrada para opcode e operando.

F03 — Unidade de Controle

Função `executarProximaInstrucao()` implementa ciclo completo.

F04 — ULA

Implementada nos casos:

- ADDK
- SUBK
- CMPK

F05 — Leitura do Sensor

Função `lerDistanciaCM()` executada pela instrução READ.

F06 — Controle de LEDs

Funções `ligarLED()` e `desligarLED()`.

F07 — Controle de Buzzer

Funções `ligarBuzzer()` e `desligarBuzzer()`.

F08 — Exibição

Instrução `DISP` mostra valor de `ACC` no Serial Monitor.

F09 — Memória Simulada

`STORE` e `LOADM` utilizam vetor `MEM[16]`.

F10 — ALERT

Avalia distância:

- <10 cm $\rightarrow$ LED + buzzer
- 10–20 cm $\rightarrow$ LED
- ≥ 20 cm $\rightarrow$ desligado

F11 — HALT

Interrompe execução:

`EXECUTANDO = false`

`MODO_RUN = false`

7. Tratamento de Overflow e Negativo

Overflow ocorre quando:

$$ACC > 9$$

Resultado negativo ocorre quando:

$$ACC < 0$$

O Serial Monitor informa explicitamente esses casos.

8. Esquema Elétrico

Componentes

- Arduino Mega 2560
- Teclado matricial 4x4
- Sensor HC-SR04
- 3 LEDs
- Buzzer ativo
- Resistores 220–330Ω
- Protoboard

Conexões principais

- Sensor TRIG → pino 41
- Sensor ECHO → pino 42
- LED1 → pino 2
- LED2 → pino 3
- LED3 → pino 4
- Buzzer → pino 45
- Teclado → pinos digitais configurados no sketch

9. Fluxo do Sistema

→ LOAD

Armazena instruções

→ Sai do LOAD

B → RUN

* → Executa 1 instrução

* → Próxima instrução

...

HALT → Encerra execução

10. Plano de Testes

Durante apresentação:

- Teste READ
- Teste ADDK / SUBK
- Teste CMPK
- Teste STORE / LOADM
- Teste LEDON / LEDOFF
- Teste BUZON / BUZOFF
- Teste ALERT
- Teste HALT

11. Conclusão

O projeto implementa uma arquitetura de computador simplificada sobre o Arduino Mega, simulando:

- Registradores
- Memória
- ISA
- Unidade de Controle
- Unidade Lógica e Aritmética
- Programa armazenado
- Ciclo de instrução

O sistema permite visualizar, na prática, os conceitos fundamentais de Arquitetura de Computadores aplicados a um sistema embarcado real.